

DSA Playbook Chapter 3: The Complete Core 75 Mastery Walkthroughs

Core Learning Objective: Master every single archetype among the canonical **Core 75** coding interview questions. Every problem features the precise problem statement, brute force analysis, core algorithmic breakthrough, optimal typed Python solution, edge-case unit assertions, runtime-generated execution trace, Big-O complexity breakdown, and follow-up interviewer variants.

Master Core 75 Architecture & Category Blueprint

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
1	Two Sum	Arrays & Hashing	Easy	Arrays & Hashing	$O(N)$	$O(N)$
2	Contains Duplicate	Arrays & Hashing	Easy	Arrays & Hashing	$O(N)$	$O(N)$
3	Valid Anagram	Arrays & Hashing	Easy	Arrays & Hashing	$O(N)$	$O(1)$
4	Group Anagrams	Arrays & Hashing	Medium	Arrays & Hashing	$O(N O(N)$	
5	Top K Frequent Elements	Arrays & Hashing	Medium	Arrays & Hashing	$O(N)$	$O(N)$
6	Product of Array Except Self	Arrays & Hashing	Medium	Arrays & Hashing	$O(N)$	$O(1)$
7	Encode and Decode Strings	Arrays & Hashing	Medium	Arrays & Hashing	$O(N)$	$O(1)$
8	Longest Consecutive Sequence	Arrays & Hashing	Medium	Arrays & Hashing	$O(N)$	$O(N)$
9	Valid Palindrome	Two Pointers	Easy	Two Pointers	$O(N)$	$O(1)$
10	3Sum	Two Pointers	Medium	Two Pointers	$O(N^2)$	$O(1)$
11	Container With Most Water	Two Pointers	Medium	Two Pointers	$O(N)$	$O(1)$
12	Trapping Rain Water	Two Pointers	Hard	Two Pointers	$O(N)$	$O(1)$
13	Best Time to Buy and Sell Stock	Sliding Window	Easy	Sliding Window	$O(N)$	$O(1)$

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
14	Longest Substring Without Repeating Characters	Sliding Window	Medium	Sliding Window	$O(N)$	$O(\min(N, \dots))$
15	Longest Repeating Character Replacement	Sliding Window	Medium	Sliding Window	$O(N)$	$O(1)$
16	Minimum Window Substring	Sliding Window	Hard	Sliding Window	$O(M O(M$	
17	Valid Parentheses	Stack	Easy	Stack	$O(N)$	$O(N)$
18	Daily Temperatures	Stack	Medium	Stack	$O(N)$	$O(N)$
19	Largest Rectangle in Histogram	Stack	Hard	Stack	$O(N)$	$O(N)$
20	Binary Search	Binary Search	Easy	Binary Search	$O(\log O(1)\$$	
21	Search in Rotated Sorted Array	Binary Search	Medium	Binary Search	$O(\log O(1)\$$	
22	Find Minimum in Rotated Sorted Array	Binary Search	Medium	Binary Search	$O(\log O(1)\$$	
23	Reverse Linked List	Linked List	Easy	Linked List	$O(N)$	$O(1)$
24	Merge Two Sorted Lists	Linked List	Easy	Linked List	$O(N O(1)\$$	
25	Reorder List	Linked List	Medium	Linked List	$O(N)$	$O(1)$

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
26	Remove Nth Node From End of List	Linked List	Medium	Linked List	$O(N)$	$O(1)$
27	Linked List Cycle	Linked List	Easy	Linked List	$O(N)$	$O(1)$
28	Merge K Sorted Lists	Linked List	Hard	Linked List	$O(N O(k))$	
29	Invert Binary Tree	Trees	Easy	Trees	$O(N)$	$O(H)$
30	Maximum Depth of Binary Tree	Trees	Easy	Trees	$O(N)$	$O(H)$
31	Same Tree	Trees	Easy	Trees	$O(N)$	$O(H)$
32	Subtree of Another Tree	Trees	Easy	Trees	$O(N O(H))$	
33	Lowest Common Ancestor of a BST	Trees	Medium	Trees	$O(H)$	$O(1)$
34	Binary Tree Level Order Traversal	Trees	Medium	Trees	$O(N)$	$O(N)$
35	Validate Binary Search Tree	Trees	Medium	Trees	$O(N)$	$O(H)$
36	Kth Smallest Element in a BST	Trees	Medium	Trees	$O(H O(H))$	
37	Construct Binary Tree from Preorder and Inorder Traversal	Trees	Medium	Trees	$O(N)$	$O(N)$

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
38	Binary Tree Maximum Path Sum	Trees	Hard	Trees	$O(N)$	$O(H)$
39	Implement Trie (Prefix Tree)	Tries	Medium	Tries	$O(L)$	$O(N)$
40	Design Add and Search Words Data Structure	Tries	Medium	Tries	$O(L)$	$O(N)$
41	Word Search II	Tries	Hard	Tries	$O(M O(\sum$	
42	Kth Largest Element in an Array	Heap / Priority Queue	Medium	Heap / Priority Queue	$O(N O(k))$	
43	Find Median from Data Stream	Heap / Priority Queue	Hard	Heap / Priority Queue	$O(\log O(N))$	
44	Combination Sum	Backtracking	Medium	Backtracking	$O(N^{\frac{T}{M}})$	$O(\frac{T}{M})$
45	Word Search	Backtracking	Medium	Backtracking	$O(M O(L))$	
46	Subsets	Backtracking	Medium	Backtracking	$O(N O(N))$	
47	Number of Islands	Graphs	Medium	Graphs	$O(M O(M$	
48	Clone Graph	Graphs	Medium	Graphs	$O(V O(V))$	
49	Pacific Atlantic Water Flow	Graphs	Medium	Graphs	$O(M O(M$	
50	Course Schedule	Graphs	Medium	Graphs	$O(V O(V$	
51	Number of Connected Components	Graphs	Medium	Graphs	$O(V O(V))$	

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
	in an Undirected Graph					
52	Graph Valid Tree	Graphs	Medium	Graphs	$O(V O(V))$	
53	Network Delay Time	Graphs	Medium	Graphs	$O(E O(V))$	
54	Alien Dictionary	Graphs	Hard	Graphs	$O(C)$	$O(U)$
55	Climbing Stairs	Dynamic Programming	Easy	Dynamic Programming	$O(N)$	$O(1)$
56	House Robber	Dynamic Programming	Medium	Dynamic Programming	$O(N)$	$O(1)$
57	House Robber II	Dynamic Programming	Medium	Dynamic Programming	$O(N)$	$O(1)$
58	Coin Change	Dynamic Programming	Medium	Dynamic Programming	$O(A O(A))$	
59	Longest Increasing Subsequence	Dynamic Programming	Medium	Dynamic Programming	$O(N O(N))$	
60	Word Break	Dynamic Programming	Medium	Dynamic Programming	$O(N O(N))$	
61	Combination Sum IV	Dynamic Programming	Medium	Dynamic Programming	$O(T O(T))$	
62	Decode Ways	Dynamic Programming	Medium	Dynamic Programming	$O(N)$	$O(N)$
63	Unique Paths	Dynamic Programming	Medium	Dynamic Programming	$O(m O(n))$	
64	Longest Common Subsequence	Dynamic Programming	Medium	Dynamic Programming	$O(M O(M))$	

#	Problem Name	Category	Difficulty	Primary Pattern	Time	Space
65	Edit Distance	Dynamic Programming	Medium	Dynamic Programming	$O(M O(M$	
66	Partition Equal Subset Sum	Dynamic Programming	Medium	Dynamic Programming	$O(N O(\text{target}))$	
67	Jump Game	Dynamic Programming	Medium	Dynamic Programming	$O(N)$	$O(1)$
68	Insert Interval	Intervals	Medium	Intervals	$O(N)$	$O(N)$
69	Merge Intervals	Intervals	Medium	Intervals	$O(N O(N)$	
70	Non-overlapping Intervals	Intervals	Medium	Intervals	$O(N O(1)$	
71	Meeting Rooms	Intervals	Easy	Intervals	$O(N O(1)$	
72	Meeting Rooms II	Intervals	Medium	Intervals	$O(N O(N)$	
73	Number of 1 Bits	Bit Manipulation	Easy	Bit Manipulation	$O(K)$	$O(1)$
74	Counting Bits	Bit Manipulation	Easy	Bit Manipulation	$O(N)$	$O(1)$
75	Missing Number	Bit Manipulation	Easy	Bit Manipulation	$O(N)$	$O(1)$

Problem 1: Two Sum (Easy - Arrays & Hashing)

Problem Statement

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. Each input has exactly one solution, and you may not use the same element twice.

Brute Force Approach & Complexity

Iterate through all pairs (i, j) with nested loops and check if `nums[i] + nums[j] == target`.
Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

As we scan each number x , the required complement is $target - x$. Store visited numbers and their indices in a hash map for $O(1)$ lookup time.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single-pass linear scan across N elements.
- **Space Complexity:** $O(N)$ to store up to N elements in the hash map.

Follow-Up Interview Variants

What if the input array is already sorted? Use Two Pointers (left = 0, right = N - 1) for $O(N)$ time and $O(1)$ auxiliary space.

Problem 2: Contains Duplicate (Easy - Arrays & Hashing)

Problem Statement

Given an integer array `nums`, return `True` if any value appears at least twice in the array, and return `False` if every element is distinct.

Brute Force Approach & Complexity

Compare every pair with nested loops ($O(N^2)$ time, $O(1)$ space) or sort the array and check adjacent elements ($O(N \log N)$ time, $O(1)$ space).

Key Insight ("Aha!" Moment)

Insert elements into a hash set. If an element is already in the set, a duplicate is found immediately in $O(1)$ average time.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ average time to scan array and insert into set.
- **Space Complexity:** $O(N)$ memory to store unique elements in the set.

Follow-Up Interview Variants

What if space must be $O(1)$ and modifying input is allowed? Sort in-place in $O(N \log N)$ time and check adjacent items.

Problem 3: Valid Anagram (Easy - Arrays & Hashing)

Problem Statement

Given two strings `s` and `t`, return `True` if `t` is an anagram of `s`, and `False` otherwise.

Brute Force Approach & Complexity

Sort both strings and compare character-by-character. Time: $O(N \log N)$, Space: $O(N)$ or $O(1)$ depending on sort implementation.

Key Insight ("Aha!" Moment)

An anagram has identical character frequency distributions. Compare frequency counters using an array of size 26 or a hash map.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ where $N = \text{len}(s)$. Single pass over both strings.
- **Space Complexity:** $O(1)$ auxiliary space since alphabet size is bounded by 26 ASCII characters ($O(k)$ for general Unicode).

Follow-Up Interview Variants

What if the inputs contain Unicode characters? Use a general hash map `dict[str, int]` instead of a fixed 26-element array.

Problem 4: Group Anagrams (Medium - Arrays & Hashing)

Problem Statement

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Brute Force Approach & Complexity

For each pair of strings, check if they are anagrams. Time: $O(N^2 \cdot K)$, Space: $O(N \cdot K)$.

Key Insight ("Aha!" Moment)

Anagrams share the exact same character frequency counts. Use a 26-character count tuple (or sorted string) as the hash map key to bucket words.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \cdot K)$ where N is number of strings and K is maximum string length.
- **Space Complexity:** $O(N \cdot K)$ to store strings grouped in the hash table.

Follow-Up Interview Variants

What if character set is arbitrary UTF-8 strings? Sort each word in $O(K \log K)$ to form the key, yielding $O(NK \log K)$ time.

Problem 5: Top K Frequent Elements (Medium - Arrays & Hashing)

Problem Statement

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Brute Force Approach & Complexity

Count frequencies with a hash map, then sort all unique items by frequency in descending order. Time: $O(N \log N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Use Bucket Sort where index represents frequency (from 0 to N). Since maximum frequency is N , filling and reverse-scanning buckets takes strictly $O(N)$ linear time.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ time using frequency bucket sorting.
- **Space Complexity:** $O(N)$ space for count hash map and frequency buckets.

Follow-Up Interview Variants

What if $k \ll N$ and data is streaming? Maintain a min-heap of size k in $O(N \log k)$ time and $O(k)$ memory.

Problem 6: Product of Array Except Self (Medium - Arrays & Hashing)

Problem Statement

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`. Must run in $O(N)$ time without using division.

Brute Force Approach & Complexity

For each index i , iterate through all other indices $j \neq i$ and multiply their values. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Any element's product except self equals (prefix product to left of i) $\times$ (suffix product to right of i). Build the result in-place using two sequential passes.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ with two linear passes.
- **Space Complexity:** $O(1)$ auxiliary memory (output array does not count toward space complexity).

Follow-Up Interview Variants

How to handle multiple zeros in the array? The prefix/suffix logic naturally handles zero, one, or multiple zeros without divide-by-zero errors.

Problem 7: Encode and Decode Strings (Medium - Arrays & Hashing)

Problem Statement

Design an algorithm to encode a list of strings to a single string, and decode that string back to the original list of strings. The strings can contain any possible characters, including delimiters like '#' and commas.

Brute Force Approach & Complexity

Using a fixed delimiter like comma or semicolon breaks immediately if strings contain that delimiter.

Key Insight ("Aha!" Moment)

Prefix each string with its length and a delimiter: `<length>#<string>`. When decoding, reading `<length>` tells exactly how many subsequent bytes belong to the word regardless of characters inside it.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ total characters processed during encode and decode.
- **Space Complexity:** $O(1)$ auxiliary space beyond storing the output strings.

Follow-Up Interview Variants

How does this protocol compare to HTTP chunked transfer encoding? It uses the exact same framing principle (hex length + CRLF + payload).

Problem 8: Longest Consecutive Sequence (Medium - Arrays & Hashing)

Problem Statement

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence. Must run in $O(N)$ time.

Brute Force Approach & Complexity

Sort the array and count longest adjacent run. Time: $O(N \log N)$, Space: $O(1)$ or $O(N)$.

Key Insight ("Aha!" Moment)

Insert all numbers into a hash set. Only begin counting a streak if `num - 1` is NOT in the set (i.e. `num` is the start of a streak). Each number is visited at most twice.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ time because each number is only checked as a sequence start once.
- **Space Complexity:** $O(N)$ memory to store unique numbers in the hash set.

Follow-Up Interview Variants

Could you use Union-Find (Disjoint Set Union) instead? Yes, union `num` with `num + 1` if present, maintaining component sizes in $O(N\alpha(N))$ time.

Problem 9: Valid Palindrome (Easy - Two Pointers)

Problem Statement

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Brute Force Approach & Complexity

Filter the string into a new cleaned string and reverse it. Time: $O(N)$, Space: $O(N)$ extra memory.

Key Insight ("Aha!" Moment)

Use two pointers starting at left and right extremes. Advance inward skipping non-alphanumeric characters, comparing lowercased characters in-place with $O(1)$ space.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass across length N .
- **Space Complexity:** $O(1)$ auxiliary space without allocating a cleaned string.

Follow-Up Interview Variants

What if you are allowed to remove at most one character to form a valid palindrome? Recursively check the two branches (`left + 1` or `right - 1`) on first mismatch.

Problem 10: 3Sum (Medium - Two Pointers)

Problem Statement

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i ≠ j`, `i ≠ k`, and `j ≠ k`, and `nums[i] + nums[j] + nums[k] = 0`. The solution set must not contain duplicate triplets.

Brute Force Approach & Complexity

Generate all triplets with 3 nested loops and store them in a set to avoid duplicates. Time: $O(N^3)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Sort the array in $O(N \log N)$. Fix element i , then use two pointers (`left = i + 1`, `right = N - 1`) to find pairs adding to `-nums[i]`. Skip duplicate adjacent elements to avoid duplicate triplets.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N^2)$ time ($O(N \log N)$ sort + N iterations of two-pointer scans).
- **Space Complexity:** $O(1)$ auxiliary space (or $O(N)$ depending on sorting algorithm implementation).

Follow-Up Interview Variants

How to extend to 4Sum or KSum? Recursively reduce KSum to $(K - 1)$ Sum until reaching 2Sum base case with two pointers in $O(N^{K-1})$.

Problem 11: Container With Most Water (Medium - Two Pointers)

Problem Statement

You are given an integer array `height` of length n . Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store.

Brute Force Approach & Complexity

Compute area for all pairs (i, j) where $\text{area} = (j - i) \times \min(h[i], h[j])$. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

The width is initially maximized by placing pointers at 0 and $N - 1$. The height is constrained by the shorter wall. Moving the taller wall can only decrease width without increasing height. Therefore, greedily advance the shorter wall inward.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass as pointers move inward until they meet.
- **Space Complexity:** $O(1)$ constant memory.

Follow-Up Interview Variants

What if you need to return the container indices instead of just area? Track the `(best_left, best_right)` pair whenever `max_water` updates.

Problem 12: Trapping Rain Water (Hard - Two Pointers)

Problem Statement

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Brute Force Approach & Complexity

For each bar i , find the maximum to its left and maximum to its right with linear scans: $\text{water}[i] = \max(0, \min(L_{\max}, R_{\max}) - h[i])$. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Water trapped at index i depends strictly on $\min(\text{max_left}, \text{max_right})$. With two pointers, whichever side has the smaller running maximum determines the water bound, allowing in-place computation with $O(1)$ space.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time; every bar is processed exactly once.
- **Space Complexity:** $O(1)$ space using two pointers.

Follow-Up Interview Variants

How to solve Trapping Rain Water in 2D (a 2D elevation grid)? Use a Min-Heap initialized with all boundary cells; pop the lowest boundary wall and traverse neighbors (Dijkstra's variant) in $O(R \cdot$

$C \log(R \cdot C)$ time.

Problem 13: Best Time to Buy and Sell Stock (Easy - Sliding Window)

Problem Statement

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day. Maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Brute Force Approach & Complexity

Try every pair (i, j) with $i < j$ and compute $\text{profit} = \text{prices}[j] - \text{prices}[i]$. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Maintain the running minimum purchase price seen so far. At each day i , the maximum profit attainable by selling on day i is $\text{prices}[i] - \text{min_price}$.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass over array.
- **Space Complexity:** $O(1)$ constant auxiliary memory.

Follow-Up Interview Variants

What if you can complete as many transactions as you like (Stock II)? Accumulate every positive adjacent price increment: $\sum \max(0, \text{prices}[i] - \text{prices}[i - 1])$.

Problem 14: Longest Substring Without Repeating Characters (Medium - Sliding Window)

Problem Statement

Given a string `s`, find the length of the longest substring without duplicate characters.

Brute Force Approach & Complexity

Generate all substrings $O(N^2)$ and check if each has unique characters $O(N)$. Time: $O(N^3)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Maintain a dynamic sliding window `[left, right]` and a hash map of characters to their most recent index. When character `s[right]` is seen inside the window, jump `left` to `last_seen[c] + 1`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time; both pointers advance strictly to the right.
- **Space Complexity:** $O(\min(N, M))$ where M is alphabet size.

Follow-Up Interview Variants

What if at most K distinct characters are permitted? Use a frequency counter hash map; shrink window while `len(counts) > K`.

Problem 15: Longest Repeating Character Replacement (Medium - Sliding Window)

Problem Statement

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character at most `k` times. Return the length of the longest substring containing the same letter you can get after performing above operations.

Brute Force Approach & Complexity

Examine every substring, count frequencies, and verify if $(\text{length} - \text{max_freq}) \leq k$. Time: $O(N^2 \cdot 26)$, Space: $O(26)$.

Key Insight ("Aha!" Moment)

A window `[left, right]` is valid if $\text{window_size} - \text{max_frequency} \leq k$. When this condition is violated, shrink the window from the left by one step.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ since both pointers only advance forward.
- **Space Complexity:** $O(1)$ auxiliary space bounded by 26 English uppercase characters.

Follow-Up Interview Variants

Do we need to decrement `max_freq` when shrinking the window? No! A new maximum window length can only be achieved if `max_freq` strictly increases beyond its previous peak.

Problem 16: Minimum Window Substring (Hard - Sliding Window)

Problem Statement

Given two strings `s` and `t` of lengths m and n respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If no such substring exists, return empty string `""`.

Brute Force Approach & Complexity

Examine every substring of s ($O(N^2)$) and check if it contains all characters of t in required frequencies ($O(N)$). Time: $O(N^3)$, Space: $O(M)$.

Key Insight ("Aha!" Moment)

Maintain required counts `need` and current window counts `have`. Keep a count `matched` of how many distinct characters satisfy their requirement. Expand `right` until `matched == len(need)`, then greedily contract `left` while maintaining validity.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(M + N)$ where $M = \text{len}(s)$ and $N = \text{len}(t)$. Each character is visited at most twice.
- **Space Complexity:** $O(M + N)$ to store character frequency maps.

Follow-Up Interview Variants

What if s contains billions of characters and can only be streamed? Keep only the filtered indices and characters that appear in t , reducing memory footprint to $O(N)$.

Problem 17: Valid Parentheses (Easy - Stack)

Problem Statement

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

Brute Force Approach & Complexity

Repeatedly replace occurrences of `"()"`, `"[]"`, and `"{}"` with `" "` until no more replacements are possible. Time: $O(N^2)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

A LIFO stack matches the most recently opened bracket with the next closing bracket. Push opening brackets; on closing brackets, pop and check if it matches.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass across length N .
- **Space Complexity:** $O(N)$ to hold opening brackets on stack in worst case.

Follow-Up Interview Variants

What if brackets include wildcards like `*` which can be `'('`, `)'`, or empty? Use two counters for minimum and maximum possible open bracket counts (Greedy Range Tracking).

Problem 18: Daily Temperatures (Medium - Stack)

Problem Statement

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the i^{th} day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] = 0`.

Brute Force Approach & Complexity

For each day i , scan ahead $j = i + 1 \dots N$ until finding a higher temperature. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Maintain a **Monotonic Decreasing Stack** storing indices of unresolved cold days. When today's temperature is warmer than the top of the stack, pop and record the elapsed days.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time; every index is pushed and popped at most once.
- **Space Complexity:** $O(N)$ auxiliary space for the stack.

Follow-Up Interview Variants

Can this be solved in $O(1)$ extra space? Iterate backwards from the end of the array, using the already-computed answers to jump forward.

Problem 19: Largest Rectangle in Histogram (Hard - Stack)

Problem Statement

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Brute Force Approach & Complexity

For each pair of bars (i, j) , find the minimum height between them and multiply by $(j - i + 1)$.
Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Maintain a **Monotonic Increasing Stack** of `(index, height)`. When a shorter bar is encountered, pop previous bars because they cannot extend further right. The popped bar's width extends between the current index and the new stack top.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time; each bar is pushed and popped at most once.
- **Space Complexity:** $O(N)$ stack memory.

Follow-Up Interview Variants

How to solve Maximal Rectangle in a 2D binary matrix? Convert each row into a histogram of consecutive 1s and run this algorithm row-by-row in $O(R \cdot C)$.

Problem 20: Binary Search (Easy - Binary Search)

Problem Statement

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

Brute Force Approach & Complexity

Linear scan through the array. Time: $O(N)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Because the array is sorted, comparing `target` with the midpoint `mid` eliminates half the remaining search space on every iteration.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(\log N)$ logarithmic time.
- **Space Complexity:** $O(1)$ constant auxiliary space.

Follow-Up Interview Variants

Why write `mid = left + (right - left) // 2` instead of `(left + right) // 2`? To avoid 32-bit integer overflow in languages like C++/Java when `left + right > 2^{31} - 1`.

Problem 21: Search in Rotated Sorted Array (Medium - Binary Search)

Problem Statement

There is an integer array `nums` sorted in ascending order (with distinct values), rotated at an unknown pivot index. Given `nums` and a `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

Brute Force Approach & Complexity

Linear search $O(N)$ across all elements. Space: $O(1)$.

Key Insight ("Aha!" Moment)

In any rotated sorted array, splitting at `mid` always leaves at least one half strictly sorted. Determine which half is sorted, check if `target` lies within that half's boundary, and bisect accordingly in $O(\log N)$.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(\log N)$ time logarithmic search.
- **Space Complexity:** $O(1)$ auxiliary memory.

Follow-Up Interview Variants

What if duplicate elements are allowed? Worst case degrades to $O(N)$ when `nums[left] == nums[mid] == nums[right]` because neither half can be guaranteed sorted.

Problem 22: Find Minimum in Rotated Sorted Array (Medium - Binary Search)

Problem Statement

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. Given the sorted rotated array `nums` of unique elements, return the minimum element of this array. Must run in $O(\log N)$ time.

Brute Force Approach & Complexity

Linear scan to find the minimum element in $O(N)$ time. Space: $O(1)$.

Key Insight ("Aha!" Moment)

Compare `nums[mid]` with `nums[right]`. If `nums[mid] > nums[right]`, the rotation pivot point (and minimum element) lies strictly to the right (`left = mid + 1`). Otherwise, the minimum is at `mid` or to the left (`right = mid`).

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(\log N)$ binary search time.
- **Space Complexity:** $O(1)$ constant space.

Follow-Up Interview Variants

How many times was the array rotated? The index of the minimum element equals the number of clockwise rotation shifts.

Problem 23: Reverse Linked List (Easy - Linked List)

Problem Statement

Given the `head` of a singly linked list, reverse the list, and return the reversed list.

Brute Force Approach & Complexity

Store all node values into an array, reverse the array, and reconstruct the linked list. Time: $O(N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Iterate through the list with three pointers: `prev`, `curr`, and `next_node`. Invert each pointer (`curr.next = prev`) in-place in $O(1)$ space.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single linear pass.
- **Space Complexity:** $O(1)$ in-place iterative pointer manipulation.

Follow-Up Interview Variants

How to implement this recursively? Base case: if `head is None or head.next is None`, return `head`. Recursive step: `new_head = reverse_list(head.next); head.next.next = head; head.next = None; return new_head` ($O(N)$ stack space).

Problem 24: Merge Two Sorted Lists (Easy - Linked List)

Problem Statement

You are given the heads of two sorted linked lists `list1` and `list2`. Merge the two lists into one sorted list by splicing together the nodes of the first two lists. Return the head of the merged linked list.

Brute Force Approach & Complexity

Dump all elements into an array, sort the array, and create a new linked list. Time: $O((N + M) \log(N + M))$, Space: $O(N + M)$.

Key Insight ("Aha!" Moment)

Use a `dummy` pre-head node and a pointer `tail`. Repeatedly attach whichever node has the smaller value (`tail.next = list1` or `list2`) in strictly $O(N + M)$ time and $O(1)$ memory.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N + M)$ where N and M are the lengths of the two lists.
- **Space Complexity:** $O(1)$ auxiliary memory by relinking existing nodes.

Follow-Up Interview Variants

What if one list is substantially longer than the other? Attach the remainder in $O(1)$ pointer assignment without looping.

Problem 25: Reorder List (Medium - Linked List)

Problem Statement

You are given the head of a singly linked-list: $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$. Reorder the list to: $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \dots$ in-place without modifying node values.

Brute Force Approach & Complexity

Store all nodes in an array for random access, then rebuild pointers with two indices from front and back. Time: $O(N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Decompose into three classic sub-problems in $O(1)$ space: 1) Find middle node with fast/slow pointers, 2) Reverse second half, 3) Interleave merge the two halves.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ total across find-mid, reverse, and merge passes.
- **Space Complexity:** $O(1)$ strictly in-place pointer mutations.

Follow-Up Interview Variants

How to test if a linked list is a Palindrome using the same building blocks? Find mid, reverse second half, compare node values from both heads, then restore list.

Problem 26: Remove Nth Node From End of List (Medium - Linked List)

Problem Statement

Given the `head` of a linked list, remove the n^{th} node from the end of the list and return its head in one single pass.

Brute Force Approach & Complexity

First pass: count length L . Second pass: advance to node $(L - n)$ and delete next node. Time: $O(N)$ (two passes), Space: $O(1)$.

Key Insight ("Aha!" Moment)

Maintain two pointers with a fixed gap of n nodes. Advance `fast` n steps ahead. Then move both `fast` and `slow` together until `fast` reaches the end; `slow` will sit directly before the node to delete.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass.
- **Space Complexity:** $O(1)$ constant auxiliary space.

Follow-Up Interview Variants

Why is a dummy node essential? It cleanly handles deleting the head node without adding a special branch condition.

Problem 27: Linked List Cycle (Easy - Linked List)

Problem Statement

Given `head`, the head of a linked list, determine if the linked list has a cycle in it using $O(1)$ memory.

Brute Force Approach & Complexity

Store visited node references in a hash set. If a node is seen again, a cycle exists. Time: $O(N)$, Space: $O(N)$ memory.

Key Insight ("Aha!" Moment)

Floyd's Tortoise and Hare algorithm: Move `slow` by 1 step and `fast` by 2 steps. If a cycle exists, `fast` catches up to `slow` by closing the gap by 1 node per iteration.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time.
- **Space Complexity:** $O(1)$ constant space without allocating memory.

Follow-Up Interview Variants

How to find the exact node where the cycle begins (Linked List Cycle II)? When `slow` and `fast` collide, reset one pointer to `head`; move both 1 step at a time; their next meeting point is the cycle start.

Problem 28: Merge K Sorted Lists (Hard - Linked List)

Problem Statement

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Brute Force Approach & Complexity

Iteratively merge lists one by one using 2-list merge. Time: $O(k \cdot N)$ where N is total number of nodes.

Key Insight ("Aha!" Moment)

Maintain a Min-Heap of size k containing the current heads of all k lists. Popping the smallest element and pushing its `next` node takes $O(\log k)$ per node, yielding $O(N \log k)$ total time.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log k)$ where N is total nodes and k is number of lists.
- **Space Complexity:** $O(k)$ heap memory holding k elements.

Follow-Up Interview Variants

Could you achieve the same complexity without a heap? Yes, using Divide and Conquer: pair up and merge k lists recursively like merge sort in $O(N \log k)$ time and $O(1)$ heap memory.

Problem 29: Invert Binary Tree (Easy - Trees)

Problem Statement

Given the `root` of a binary tree, invert the tree (mirror reflection), and return its root.

Brute Force Approach & Complexity

Reconstruct tree using level-order serialization mirrored. Time: $O(N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Post-order or pre-order recursive traversal: for every node, swap its left and right child pointers, then recursively invert both subtrees.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time visiting all N nodes.
- **Space Complexity:** $O(H)$ recursion stack space where H is tree height ($O(\log N)$ balanced, $O(N)$ skewed).

Follow-Up Interview Variants

How to solve this iteratively without recursion? Use a BFS queue: pop node, swap its children, and push both non-null children onto the queue.

Problem 30: Maximum Depth of Binary Tree (Easy - Trees)

Problem Statement

Given the `root` of a binary tree, return its maximum depth (number of nodes along the longest path from root to farthest leaf).

Brute Force Approach & Complexity

Find all root-to-leaf paths and take the maximum length. Time: $O(N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Depth of node N equals $1 + \max(\text{depth}(N.\text{left}), \text{depth}(N.\text{right}))$. Base case: depth of `None` is 0.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ visiting every node once.
- **Space Complexity:** $O(H)$ recursion call stack space.

Follow-Up Interview Variants

How to calculate minimum depth? Be careful with single-child nodes: if one child is `None`, depth is determined by the non-null child, not `min(0, right)`.

Problem 31: Same Tree (Easy - Trees)

Problem Statement

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not. Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Brute Force Approach & Complexity

Serialize both trees with null markers into strings and compare. Time: $O(N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Simultaneous DFS traversal: if both nodes are `None`, return `True`; if one is `None` or values differ, return `False`; recursively check both `(p.left, q.left)` and `(p.right, q.right)`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ visiting every node at most once.
- **Space Complexity:** $O(H)$ recursion stack space.

Follow-Up Interview Variants

How to test if two trees are Symmetric Mirror Trees? Compare `(p.left, q.right)` and `(p.right, q.left)` simultaneously.

Problem 32: Subtree of Another Tree (Easy - Trees)

Problem Statement

Given the roots of two binary trees `root` and `subRoot`, return `True` if there is a subtree of `root` with the same structure and node values of `subRoot` and `False` otherwise.

Brute Force Approach & Complexity

For every node in `root`, check if the subtree starting there is identical to `subRoot` using `is_same_tree`. Time: $O(N \cdot M)$, Space: $O(H)$.

Key Insight ("Aha!" Moment)

Traverse `root` with DFS. If `is_same_tree(root, subRoot)` is true, return `True`; otherwise, search recursively in `root.left` or `root.right`.

Optimal Python Solution

Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N \cdot M)$ where N and M are node counts.
- **Space Complexity:** $O(H)$ recursion call stack.

Follow-Up Interview Variants

How to achieve $O(N + M)$ strictly linear time? Serialize both trees using pre-order traversal with distinct null and delimiter tokens, then run KMP (Knuth-Morris-Pratt) string search.

Problem 33: Lowest Common Ancestor of a BST (Medium - Trees)

Problem Statement

Given a Binary Search Tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

Brute Force Approach & Complexity

Find root-to-node paths for both nodes and compare paths to find divergence point. Time: $O(N)$, Space: $O(H)$.

Key Insight ("Aha!" Moment)

Exploit BST ordering property: if both values are less than `curr.val`, LCA must be in left subtree; if both are greater, LCA must be in right subtree. The moment they split (or one equals `curr.val`), `curr` is the LCA!

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(H)$ where H is BST height ($O(\log N)$ balanced).
- **Space Complexity:** $O(1)$ iterative traversal.

Follow-Up Interview Variants

What if the tree is an arbitrary Binary Tree (not a BST)? Use post-order DFS: return node if it matches p or q ; if both left and right return non-null, current node is LCA.

Problem 34: Binary Tree Level Order Traversal (Medium - Trees)

Problem Statement

Given the `root` of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

Brute Force Approach & Complexity

Calculate depth of tree, then for each depth d , traverse from root collecting nodes at depth d .
Time: $O(N^2)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Use a FIFO Queue (BFS). In each iteration of the outer loop, record `level_size = len(queue)` and process exactly that many nodes to group them by layer.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ visiting every node exactly once.
- **Space Complexity:** $O(N)$ queue holds at most $N/2$ nodes at bottom level.

Follow-Up Interview Variants

How to output zigzag level order traversal (alternating left-right and right-left)? Reverse alternate level lists or append to a double-ended queue using a parity flag.

Problem 35: Validate Binary Search Tree (Medium - Trees)

Problem Statement

Given the `root` of a binary tree, determine if it is a valid binary search tree (BST). A valid BST requires all left subtree values to be strictly less than node value, and all right subtree values strictly greater.

Brute Force Approach & Complexity

Only checking `node.left.val < node.val < node.right.val` locally fails because a node deep in the left subtree could violate the root's value.

Key Insight ("Aha!" Moment)

Pass valid value boundaries `(low, high)` down the DFS recursion. When going left, update `high = node.val`; when going right, update `low = node.val`.

Optimal Python Solution

```
def isValidBST(self, root: Optional[TreeNode]) -> bool:
    def dfs(node: Optional[TreeNode], low: int, high: int) -> bool:
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)
    return dfs(root, float('-inf'), float('inf'))
```

Edge-Case Asserts

```
assert isValidBST(TreeNode(1)) == True
assert isValidBST(TreeNode(2, 1)) == True
assert isValidBST(TreeNode(1, 2)) == False
assert isValidBST(TreeNode(2, 1, 3)) == True
assert isValidBST(TreeNode(1, 2, 3)) == False
```

Dry-Run Execution Trace

```
def dfs(node, low, high):
    if not node: return True
    if not (low < node.val < high): return False
    return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)

# Trace for root=2, left=1, right=3
# dfs(2, -inf, inf) -> True
#   dfs(1, -inf, 2) -> True
#   dfs(3, 2, inf) -> True
```

Complexity Analysis

- **Time Complexity:** $O(N)$ visiting every node once.
- **Space Complexity:** $O(H)$ recursion call stack space.

Follow-Up Interview Variants

Could this be verified using in-order traversal? Yes, an in-order traversal of a valid BST must produce a strictly increasing sequence with $prev < curr$.

Problem 36: Kth Smallest Element in a BST (Medium - Trees)

Problem Statement

Given the `root` of a binary search tree, and an integer `k`, return the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.

Brute Force Approach & Complexity

Dump all node values into an array, sort it, and return index $k - 1$. Time: $O(N \log N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

In-order traversal (`Left → Root → Right`) of a BST yields keys in strictly sorted ascending order. Stop early as soon as the k^{th} node is visited.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(H + k)$ time where H is tree height.
- **Space Complexity:** $O(H)$ stack space.

Follow-Up Interview Variants

What if the BST is modified frequently (often searched and updated)? Store the count of nodes in the left subtree at each node (Order Statistic Tree), enabling $O(H)$ lookups.

Problem 37: Construct Binary Tree from Preorder and Inorder Traversal (Medium - Trees)

Problem Statement

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return the binary tree.

Brute Force Approach & Complexity

Recursively search for root in `inorder` using linear scans: $O(N^2)$ time.

Key Insight ("Aha!" Moment)

The first element of `preorder` is always the root. Locating this root in `inorder` splits elements into left and right subtrees. Cache `inorder` index locations in a hash map for $O(1)$ lookups, yielding $O(N)$ total time.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time to build N nodes.
- **Space Complexity:** $O(N)$ for hash map and recursion stack.

Follow-Up Interview Variants

Can a binary tree be uniquely constructed from Preorder and Postorder traversals? Not in general (unless every non-leaf node has exactly two children).

Problem 38: Binary Tree Maximum Path Sum (Hard - Trees)

Problem Statement

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes has an edge connecting them. Return the maximum path sum of any non-empty path.

Brute Force Approach & Complexity

Find all pairs of nodes and sum values along the path between them. Time: $O(N^2)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

Post-order Tree DP: at each node N , compute $\text{left_gain} = \max(0, \text{dfs}(N.\text{left}))$ and $\text{right_gain} = \max(0, \text{dfs}(N.\text{right}))$. The local peak path is $N.\text{val} + \text{left_gain} + \text{right_gain}$. Return $N.\text{val} + \max(\text{left_gain}, \text{right_gain})$ to the caller.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ visiting every node once.
- **Space Complexity:** $O(H)$ recursion call stack space.

Follow-Up Interview Variants

What if all values in the tree are negative? Clamping gains to 0 allows the algorithm to pick the single largest negative node rather than accumulating negative sums.

Problem 39: Implement Trie (Prefix Tree) (Medium - Tries)

Problem Statement

A trie (pronounced 'try') or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. Implement `insert`, `search`, and `starts_with` methods.

Brute Force Approach & Complexity

Store strings in a list: `insert` is $O(1)$, but `search` and `starts_with` require $O(N \cdot L)$ scans across all words.

Key Insight ("Aha!" Moment)

Each node contains a map of child nodes `children: dict[str, TrieNode]` and boolean `is_end`. Traversal follows character edges in $O(L)$ time independent of the number of words stored.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(L)$ for each operation where L is word length.
- **Space Complexity:** $O(N \cdot L)$ in worst case with no shared prefixes.

Follow-Up Interview Variants

How to optimize memory in memory-constrained environments? Use a Radix Tree (Patricia Trie) which compacts non-branching paths into single multi-character edges.

Problem 40: Design Add and Search Words Data Structure (Medium - Tries)

Problem Statement

Design a data structure that supports adding new words and finding if a string matches any previously added string, where `.` can represent any letter.

Brute Force Approach & Complexity

Maintain a list and test with regular expressions on every search query. Time: $O(N \cdot L)$ per search.

Key Insight ("Aha!" Moment)

Store words in a Trie. For exact characters, traverse the specific child node. When encountering `.`, branch DFS across all child nodes at the current level.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(L)$ for `add_word`; $O(26^L)$ worst-case for search with all dots, $O(L)$ average case.
- **Space Complexity:** $O(N \cdot L)$ to store words in the Trie.

Follow-Up Interview Variants

How to prevent worst-case exponential slowdown on searches like `"...."`? Group words by length in separate Tries or hash maps.

Problem 41: Word Search II (Hard - Tries)

Problem Statement

Given an $m \times n$ `board` of characters and a list of strings `words`, return all words on the board. Each word must be constructed from sequentially adjacent cells.

Brute Force Approach & Complexity

Run standard Word Search I for each word in `words`. Time: $O(W \cdot M \cdot N \cdot 4^L)$.

Key Insight ("Aha!" Moment)

Build a Trie of all target words. Run DFS on the board once, traversing the Trie simultaneously. Prune leaf nodes from the Trie when a word is discovered to optimize subsequent searches.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(M \cdot N \cdot 4^L)$ where L is maximum word length.
- **Space Complexity:** $O(\sum L)$ space to store all words in Trie.

Follow-Up Interview Variants

Why is leaf pruning crucial? Once a leaf word is found, pruning that path prevents future DFS explorations from retreading dead-end branches.

Problem 42: Kth Largest Element in an Array (Medium - Heap / Priority Queue)

Problem Statement

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array. Note that it is the k^{th} largest element in sorted order, not the k^{th} distinct element.

Brute Force Approach & Complexity

Sort the entire array in descending order and return `nums[k-1]`. Time: $O(N \log N)$, Space: $O(1)$ or $O(N)$.

Key Insight ("Aha!" Moment)

Maintain a **Min-Heap of size k** . As elements are processed, push to heap and pop smallest when size exceeds k . At the end, the root of the min-heap holds the k^{th} largest element.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log k)$ maintaining a size- k heap.
- **Space Complexity:** $O(k)$ auxiliary space for the heap.

Follow-Up Interview Variants

Can this be solved in $O(N)$ average time? Yes, using QuickSelect (Hoare's selection algorithm) with randomized partitioning.

Problem 44: Combination Sum (Medium - Backtracking)

Problem Statement

Given an array of distinct integers `candidates` and a target integer `target`, return a list of all unique combinations of `candidates` where the chosen numbers sum to `target`. The same number may be chosen unlimited times.

Brute Force Approach & Complexity

Generate all possible multisets up to target length and check sums. Exponential: $O(2^N)$ or worse.

Key Insight ("Aha!" Moment)

Depth-First Search (DFS) Backtracking: at index i , either include `candidates[i]` (staying at index i to allow reuse) or advance to $i + 1$ (exclude `candidates[i]`). Prune when `current_sum > target`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N^{\frac{T}{M}})$ where T is target and M is minimal candidate value.
- **Space Complexity:** $O(\frac{T}{M})$ recursion stack depth.

Follow-Up Interview Variants

What if each number can be used at most once and duplicates exist in candidates (Combination Sum II)? Sort candidates and skip adjacent duplicates: `if i > start and candidates[i] == candidates[i-1]: continue`.

Problem 45: Word Search (Medium - Backtracking)

Problem Statement

Given an $m \times n$ grid of characters `board` and a string `word`, return `True` if `word` exists in the grid. The word can be constructed from sequentially adjacent cells horizontally or vertically. The same cell may not be used more than once.

Brute Force Approach & Complexity

Generate all self-avoiding paths of length L in the grid: $O(M \cdot N \cdot 4^L)$.

Key Insight ("Aha!" Moment)

Explore with Backtracking DFS. In-place mark visited cell by temporarily setting `board[r][c] = '#'` and restoring it on backtracking. Terminate immediately upon full word match.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(M \cdot N \cdot 3^L)$ because after the initial step we do not turn backwards.
- **Space Complexity:** $O(L)$ recursion call stack space where L is word length.

Follow-Up Interview Variants

How to prune searches before starting? Count character frequencies in the board vs `word`. If the board lacks sufficient frequency for any character, return `False` immediately.

Problem 46: Subsets (Medium - Backtracking)

Problem Statement

Given an integer array `nums` of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Brute Force Approach & Complexity

Iterate through integers 0 to $2^N - 1$, taking the j^{th} element if the j^{th} bit is set. Time: $O(N \cdot 2^N)$, Space: $O(N \cdot 2^N)$.

Key Insight ("Aha!" Moment)

At each step in Backtracking DFS, either include `nums[i]` or exclude `nums[i]`, generating all 2^N subsets systematically.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \cdot 2^N)$ to generate 2^N subsets of average length $N/2$.
- **Space Complexity:** $O(N)$ recursion depth.

Follow-Up Interview Variants

What if the input contains duplicates (Subsets II)? Sort the array first and skip duplicate branches when excluding the current element.

Problem 47: Number of Islands (Medium - Graphs)

Problem Statement

Given an $m \times n$ 2D binary grid `grid` which represents a map of `'1'` s (land) and `'0'` s (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

Brute Force Approach & Complexity

Find all land cells and compute connected components via an $O((M \cdot N)^2)$ all-pairs reachability scan.

Key Insight ("Aha!" Moment)

Iterate through grid. Whenever a `'1'` is found, increment island count and flood-fill sink all connected land cells to `'0'` using BFS or DFS.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(M \cdot N)$ visiting every cell a constant number of times.
- **Space Complexity:** $O(M \cdot N)$ worst-case call stack space for a grid filled with land.

Follow-Up Interview Variants

What if mutating the input grid is forbidden? Maintain a `visited` boolean set/matrix or use Disjoint Set Union (Union-Find).

Problem 48: Clone Graph (Medium - Graphs)

Problem Statement

Given a reference of a node in a connected undirected graph, return a deep copy (clone) of the graph. Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

Brute Force Approach & Complexity

Traversing without tracking visited nodes causes an infinite loop due to graph cycles.

Key Insight ("Aha!" Moment)

Maintain a hash map `visited: dict[Node, Node]` mapping original node references to their cloned copies. For each neighbor, either recurse or return the existing clone from the map.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(V + E)$ visiting every vertex and edge once.
- **Space Complexity:** $O(V)$ hash map memory to store vertex clones.

Follow-Up Interview Variants

How to implement this with BFS instead of DFS? Use a queue: pop node, iterate neighbors, clone unvisited neighbors, push to queue, and wire up neighbor pointers.

Problem 49: Pacific Atlantic Water Flow (Medium - Graphs)

Problem Statement

There is an $m \times n$ rectangular island that borders both the Pacific Ocean (top and left edges) and Atlantic Ocean (bottom and right edges). Water flows from a cell to an adjacent cell with an equal

or lower height. Return a list of grid coordinates where water can flow to both oceans.

Brute Force Approach & Complexity

Run BFS/DFS from every cell (r, c) to check if both oceans are reachable. Time: $O((M \cdot N)^2)$.

Key Insight ("Aha!" Moment)

Reverse the problem: start from the ocean borders and simulate water flowing **uphill** ($h[\text{next}] \geq h[\text{curr}]$). Cells in the intersection of both reachability sets flow to both oceans.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(M \cdot N)$ as each cell is visited at most twice.
- **Space Complexity:** $O(M \cdot N)$ memory for reachability sets.

Follow-Up Interview Variants

What if water can flow diagonally as well? Add 4 diagonal vectors to the exploration directions without changing the algorithmic structure.

Problem 50: Course Schedule (Medium - Graphs)

Problem Statement

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a, b]` indicates that you must take course `b` first if you want to take course `a`. Return `True` if you can finish all courses.

Brute Force Approach & Complexity

Exhaustively check all permutations of course sequences: $O(N!)$.

Key Insight ("Aha!" Moment)

This is directed graph cycle detection. Use **Kahn's Algorithm (BFS with In-Degrees)**: start with in-degree 0 nodes; decrement neighbor in-degrees on pop. If the number of processed nodes equals `numCourses`, no cycle exists.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(V + E)$ where $V = \text{numCourses}$ and $E = \text{len}(\text{prerequisites})$.
- **Space Complexity:** $O(V + E)$ adjacency list and in-degree table.

Follow-Up Interview Variants

How to return the valid course sequence order (Course Schedule II)? Append each popped node to an `order` list; return `order` if `len(order) == numCourses` else `[]`.

Problem 51: Number of Connected Components in an Undirected Graph (Medium - Graphs)

Problem Statement

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [a, b]` indicates that there is an edge between a and b in the graph. Return the number of connected components in the graph.

Brute Force Approach & Complexity

Adjacency list with DFS from every unvisited node. Time: $O(V + E)$, Space: $O(V + E)$.

Key Insight ("Aha!" Moment)

Use **Disjoint Set Union (DSU)** with path compression and union-by-rank. Start with n components; each successful union of two disjoint roots decrements the component count by 1 in nearly $O(1)$ amortized time ($O(\alpha(N))$).

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(V + E \cdot \alpha(V))$ nearly linear time where α is inverse Ackermann function.
- **Space Complexity:** $O(V)$ parent and rank arrays.

Follow-Up Interview Variants

When is DSU preferable over BFS/DFS? DSU excels in dynamic connectivity where edges arrive continuously over time (online graph connectivity).

Problem 52: Graph Valid Tree (Medium - Graphs)

Problem Statement

Given n nodes labeled from 0 to $n - 1$ and a list of undirected edges, write a function to check whether these edges make up a valid tree.

Brute Force Approach & Complexity

DFS to check if graph is connected and has no cycles. Time: $O(V + E)$.

Key Insight ("Aha!" Moment)

A graph of n nodes is a valid tree if and only if: 1) It has exactly $n - 1$ edges, and 2) It is fully connected (no cycles). With Union-Find, if `find(u) == find(v)` on any edge, a cycle exists.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(V \cdot \alpha(V))$ amortized time.
- **Space Complexity:** $O(V)$ parent array memory.

Follow-Up Interview Variants

Why is `len(edges) == n - 1` an essential initial check? It immediately filters out under-connected graphs ($E < n - 1$) and graphs that must contain cycles ($E > n - 1$).

Problem 53: Network Delay Time (Medium - Graphs)

Problem Statement

You are given a network of n nodes, labeled from 1 to n . You are also given `times`, a list of travel times as directed edges `times[i] = (u, v, w)`. We will send a signal from node k . Return the minimum time it takes for all n nodes to receive the signal. If it is impossible, return `-1`.

Brute Force Approach & Complexity

Bellman-Ford algorithm with V iterations: $O(V \cdot E)$.

Key Insight ("Aha!" Moment)

Use **Dijkstra's Shortest Path Algorithm** with a Min-Heap: track shortest confirmed distance to each node. Pop lowest-latency frontier node, relax outgoing edges, and return the maximum distance among all nodes.

Optimal Python Solution

Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(E \log V)$ using binary min-heap.
- **Space Complexity:** $O(V + E)$ graph and heap storage.

Follow-Up Interview Variants

What if edge weights could be negative? Dijkstra fails with negative weights; you must use Bellman-Ford or SPFA in $O(V \cdot E)$ time.

Problem 54: Alien Dictionary (Hard - Graphs)

Problem Statement

There is a new alien language that uses the English alphabet. Given a list of words from the alien language dictionary sorted lexicographically by the rules of this new language, derive the order of letters in this language. If the order is invalid, return `""`.

Brute Force Approach & Complexity

Compare all pairs of words and check all permutations: factorial complexity.

Key Insight ("Aha!" Moment)

Compare adjacent pairs of words `words[i]` and `words[i+1]`. The first differing character gives a directed edge $c_1 \rightarrow c_2$. Build graph and run Topological Sort (Kahn's or post-order DFS). If a cycle occurs (or prefix rule is violated like `"abc"` before `"ab"`), return `""`.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(C)$ where C is total length of all words combined.
- **Space Complexity:** $O(U + E)$ where U is unique characters (at most 26).

Follow-Up Interview Variants

What if multiple valid alien alphabets exist? Any valid topological sort satisfies the problem criteria.

Problem 55: Climbing Stairs (Easy - Dynamic Programming)

Problem Statement

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Brute Force Approach & Complexity

Recursively compute `climb(n-1) + climb(n-2)` without memoization. Time: $O(2^N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

To reach step n , you must have come from step $n - 1$ (taking 1 step) or step $n - 2$ (taking 2 steps). Hence $W(n) = W(n - 1) + W(n - 2)$ (Fibonacci sequence) computed in $O(1)$ space.

Optimal Python Solution



Edge-Case Asserts



Dry-Run Execution Trace



Complexity Analysis

- **Time Complexity:** $O(N)$ linear time.
- **Space Complexity:** $O(1)$ constant auxiliary memory.

Follow-Up Interview Variants

Can this be solved in $O(\log N)$ time? Yes, using Matrix Exponentiation on $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$ or Binet's Fibonacci formula.

Problem 56: House Robber (Medium - Dynamic Programming)

Problem Statement

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. Adjacent houses have security systems connected and will automatically contact the police if two adjacent houses were broken into on the same night. Determine the maximum amount of money you can rob tonight without alerting the police.

Brute Force Approach & Complexity

Generate all non-adjacent subsets and take the maximum sum. Time: $O(2^N)$, Space: $O(N)$.

Key Insight ("Aha!" Moment)

At house i , choose either: 1) Rob house i plus max loot from house $i - 2$, or 2) Skip house i and keep max loot from house $i - 1$. Rolling state `rob1, rob2` achieves $O(1)$ space.

Optimal Python Solution

```
def rob(nums):
    if not nums:
        return 0
    rob1, rob2 = 0, 0
    for i in range(len(nums)):
        rob1, rob2 = rob2, max(rob1 + nums[i], rob2)
    return rob2
```

Edge-Case Asserts

```
assert rob([0]) == 0
assert rob([1]) == 1
assert rob([2, 1]) == 2
assert rob([1, 2, 3]) == 3
assert rob([1, 2, 3, 1]) == 3
assert rob([1, 2, 3, 4, 5]) == 7
```

Dry-Run Execution Trace

```
nums = [1, 2, 3, 4, 5]
i=0: rob1=0, rob2=0 -> rob2=1, rob1=0
i=1: rob1=1, rob2=0 -> rob2=max(1+2, 0)=3, rob1=1
i=2: rob1=3, rob2=1 -> rob2=max(3+3, 1)=6, rob1=3
i=3: rob1=6, rob2=3 -> rob2=max(6+4, 3)=10, rob1=6
i=4: rob1=10, rob2=6 -> rob2=max(10+5, 6)=15, rob1=10
return 15
```

Complexity Analysis

- **Time Complexity:** $O(N)$ linear single pass.

- **Space Complexity:** $O(1)$ rolling variables.

Follow-Up Interview Variants

What if houses are arranged in a tree structure (House Robber III)? Return a tuple `(rob_root, skip_root)` using post-order tree DP.

Problem 57: House Robber II (Medium - Dynamic Programming)

Problem Statement

You are a professional robber planning to rob houses along a street, but all houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Determine the maximum amount of money you can rob tonight without alerting the police.

Brute Force Approach & Complexity

Exhaustively test all valid combinations: $O(2^N)$.

Key Insight ("Aha!" Moment)

Since the first and last houses are adjacent, you can never rob both. Break the circle into two linear sub-problems: `rob(nums[:-1])` (exclude last house) and `rob(nums[1:])` (exclude first house), taking $\max(\text{Case A}, \text{Case B})$.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ two linear passes.
- **Space Complexity:** $O(1)$ constant auxiliary memory.

Follow-Up Interview Variants

What if houses are arranged in a 2D grid where adjacent houses share an edge? This becomes the Maximum Independent Set on bipartite grid graphs, solvable via Max Flow / Min Cut.

Problem 58: Coin Change (Medium - Dynamic Programming)

Problem Statement

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

Brute Force Approach & Complexity

Exhaustively explore all combinations with recursive branching: $O(S^A)$ where S is coin count and A is amount.

Key Insight ("Aha!" Moment)

Bottom-up Unbounded Knapsack: `dp[a]` stores minimum coins for amount a . Transition: `dp[a] = min(dp[a], 1 + dp[a - c])` for each coin $c \leq a$.

Optimal Python Solution

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for a in range(coin, amount + 1):
            dp[a] = min(dp[a], 1 + dp[a - coin])
    return dp[amount] if dp[amount] != float('inf') else -1
```

Edge-Case Asserts

```
assert coinChange([1], 0) == 0
assert coinChange([2], 3) == -1
assert coinChange([1, 2, 5], 11) == 3
```

Dry-Run Execution Trace

```
coins = [1, 2, 5], amount = 11
dp = [inf, inf, inf, inf, inf, inf, inf, inf, inf, inf, inf, inf]
dp[0] = 0
coin = 1: dp = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
coin = 2: dp = [0, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6]
coin = 5: dp = [0, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4]
```

Complexity Analysis

- **Time Complexity:** $O(A \cdot C)$ where A is target amount and C is number of coin denominations.
- **Space Complexity:** $O(A)$ array of size `amount + 1`.

Follow-Up Interview Variants

How to count the number of UNIQUE combinations that sum to amount (Coin Change II)? Iterate coins on outer loop and amounts on inner loop: `dp[a] += dp[a - c]`.

Problem 59: Longest Increasing Subsequence (Medium - Dynamic Programming)

Problem Statement

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Brute Force Approach & Complexity

Classic $O(N^2)$ DP where `dp[i] = 1 + max(dp[j])` for all $j < i$ with `nums[j] < nums[i]`.

Key Insight ("Aha!" Moment)

Patience Sorting with Binary Search: maintain array `tails` where `tails[i]` is the smallest tail element of all increasing subsequences of length $i + 1$. For each `num`, binary search its insertion position in $O(\log N)$, yielding $O(N \log N)$ total time.

Optimal Python Solution

```
def lengthOfLongestIncreasingSubsequence(nums):
    tails = []
    for num in nums:
        # Binary search for the insertion point
        left, right = 0, len(tails)
        while left < right:
            mid = (left + right) // 2
            if tails[mid] < num:
                left = mid + 1
            else:
                right = mid
        # Insert the number at the found position
        if left == len(tails):
            tails.append(num)
        else:
            tails[left] = num
    return len(tails)
```

Edge-Case Asserts

```
assert lengthOfLongestIncreasingSubsequence([1]) == 1
assert lengthOfLongestIncreasingSubsequence([2, 1]) == 1
assert lengthOfLongestIncreasingSubsequence([1, 2, 3]) == 3
assert lengthOfLongestIncreasingSubsequence([1, 3, 2]) == 2
```

Dry-Run Execution Trace

```
nums = [10, 9, 2, 5, 3, 7, 101, 18]
tails = []
for num in nums:
    # Binary search for the insertion point
    left, right = 0, len(tails)
    while left < right:
        mid = (left + right) // 2
        if tails[mid] < num:
            left = mid + 1
        else:
            right = mid
    # Insert the number at the found position
    if left == len(tails):
        tails.append(num)
    else:
        tails[left] = num
    print(f"num={num}, tails={tails}")
print(f"Final tails: {tails}, length: {len(tails)}")
```

Complexity Analysis

- **Time Complexity:** $O(N \log N)$ time using binary search bisection.
- **Space Complexity:** $O(N)$ space for tails array.

Follow-Up Interview Variants

How to reconstruct the actual subsequence instead of just its length? Maintain predecessor pointers array `parent` mapping each element to its previous subsequence item.

Problem 60: Word Break (Medium - Dynamic Programming)

Problem Statement

Given a string `s` and a dictionary of strings `wordDict`, return `True` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Brute Force Approach & Complexity

Recursively check every prefix and test remainder: $O(2^N)$ time.

Key Insight ("Aha!" Moment)

1D Dynamic Programming: `dp[i]` indicates whether prefix `s[:i]` can be segmented. For every index i and word w in dictionary, if `dp[i - len(w)]` is `True` and `s[i - len(w) : i] == w`, then `dp[i] = True`.

Optimal Python Solution

```
def wordBreak(s: str, wordDict: List[str]) -> bool:
    wordSet = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True
    return dp[-1]
```

Edge-Case Asserts

```
assert wordBreak("leetcode", ["leet", "code"]) == True
assert wordBreak("applepenapple", ["apple", "pen"]) == True
assert wordBreak("catsanddog", ["cat", "and", "dog", "sand"]) == False
```

Dry-Run Execution Trace

```
Input: s = "leetcode", wordDict = ["leet", "code"]
dp = [True, False, False, False, False, False, False, False, False, False]
i = 1: j = 0, s[0:1] = "l" not in wordDict
i = 2: j = 0, s[0:2] = "le" not in wordDict; j = 1, s[1:2] = "e" not in wordDict
i = 3: j = 0, s[0:3] = "lee" not in wordDict; j = 1, s[1:3] = "et" not in wordDict
i = 4: j = 0, s[0:4] = "leet" in wordDict -> dp[4] = True; j = 1, s[1:4] = "eet" not in wordDict
i = 5: j = 0, s[0:5] = "leetc" not in wordDict; j = 1, s[1:5] = "eetc" not in wordDict; j = 2, s[2:5] = "etc" not in wordDict
i = 6: j = 0, s[0:6] = "leetcode" not in wordDict; j = 1, s[1:6] = "eetcode" not in wordDict; j = 2, s[2:6] = "etcode" not in wordDict; j = 3, s[3:6] = "code" in wordDict -> dp[6] = True
Output: True
```

Complexity Analysis

- **Time Complexity:** $O(N \cdot M \cdot K)$ where $N = \text{len}(s)$, M is dictionary word count, and K is max word length.
- **Space Complexity:** $O(N)$ for the boolean DP array.

Follow-Up Interview Variants

How to return all possible segmented sentences (Word Break II)? Use DFS with memoization returning list of formed sentences.

Problem 61: Combination Sum IV (Medium - Dynamic Programming)

Problem Statement

Given an array of distinct integers `nums` and a target integer `target`, return the number of possible combinations (permutations) that add up to `target`.

Brute Force Approach & Complexity

Recursively branch over all elements on every step: $O(N^T)$.

Key Insight ("Aha!" Moment)

Because order matters (e.g. `(1, 2)` and `(2, 1)` are distinct), this is a permutation DP. Transition: `dp[i] = sum(dp[i - num] for num in nums if i ≥ num)` with base case `dp[0] = 1`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(T \cdot N)$ where T is target and N is array length.
- **Space Complexity:** $O(T)$ DP array of size $T + 1$.

Follow-Up Interview Variants

What if negative numbers were allowed in `nums`? Cycles of arbitrary length could sum to zero, creating infinite combinations. A maximum path length limit would be required.

Problem 62: Decode Ways (Medium - Dynamic Programming)

Problem Statement

A message containing letters from A-Z can be encoded into numbers using the mapping 'A' → 1, 'B' → 2, ... 'Z' → 26. Given a string `s` containing only digits, return the number of ways to decode it.

Brute Force Approach & Complexity

Explore all 1-character and 2-character splits recursively: $O(2^N)$ time.

Key Insight ("Aha!" Moment)

At index i , check two transitions: 1) Single digit $s[i - 1]$ (valid if $\neq '0'$), adding $dp[i - 1]$; 2) Two digits $s[i - 2 : i]$ (valid if between 10 and 26), adding $dp[i - 2]$.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single linear pass.
- **Space Complexity:** $O(N)$ or $O(1)$ by storing only the previous two states.

Follow-Up Interview Variants

What if `*` characters represent any digit from 1 to 9 (Decode Ways II)? Branch cases taking into account possibilities for `*` alone (9 ways) and paired with adjacent digits.

Problem 63: Unique Paths (Medium - Dynamic Programming)

Problem Statement

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner and tries to move to the bottom-right corner. The robot can only move either down or right at any point in time. Return the number of possible unique paths.

Brute Force Approach & Complexity

Recursive DFS moving down or right: $O(2^{m+n})$.

Key Insight ("Aha!" Moment)

To reach cell (r, c) , the robot comes from above $(r - 1, c)$ or left $(r, c - 1)$. Transition: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`. Rolling 1D array achieves $O(n)$ space.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(m \cdot n)$ filling the table.
- **Space Complexity:** $O(n)$ space using a rolling single row.

Follow-Up Interview Variants

Can this be computed in $O(m)$ without dynamic programming? Yes, using combinatorics: the robot must make exactly $(m - 1)$ down steps and $(n - 1)$ right steps, which equals $\binom{m+n-2}{m-1}$.

Problem 64: Longest Common Subsequence (Medium - Dynamic Programming)

Problem Statement

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return `0`.

Brute Force Approach & Complexity

Generate all 2^M subsequences of `text1` and check if they are in `text2`. Time: $O(2^M \cdot N)$.

Key Insight ("Aha!" Moment)

2D Dynamic Programming: if characters match (`text1[i] == text2[j]`), `dp[i][j] = 1 + dp[i-1][j-1]`. Otherwise, discard one character from either string: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.

Optimal Python Solution

```
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Edge-Case Asserts

```
assert longestCommonSubsequence("abc", "") == 0
assert longestCommonSubsequence("", "abc") == 0
assert longestCommonSubsequence("a", "a") == 1
assert longestCommonSubsequence("abc", "abc") == 3
assert longestCommonSubsequence("abc", "bca") == 2
```

Dry-Run Execution Trace

```
text1 = "abc", text2 = "bca"
m, n = 3, 3
dp = [[0] * 4 for _ in range(4)]
# i=1, j=1: 'a' != 'b', dp[1][1] = max(dp[0][1], dp[1][0]) = 0
# i=1, j=2: 'a' != 'c', dp[1][2] = max(dp[0][2], dp[1][1]) = 0
# i=1, j=3: 'a' != 'a', dp[1][3] = max(dp[0][3], dp[1][2]) = 0
# i=2, j=1: 'b' == 'b', dp[2][1] = dp[1][0] + 1 = 1
# i=2, j=2: 'b' != 'c', dp[2][2] = max(dp[1][2], dp[2][1]) = 1
# i=2, j=3: 'b' != 'a', dp[2][3] = max(dp[1][3], dp[2][2]) = 1
# i=3, j=1: 'c' != 'b', dp[3][1] = max(dp[2][1], dp[3][0]) = 1
# i=3, j=2: 'c' != 'c', dp[3][2] = max(dp[2][2], dp[3][1]) = 1
# i=3, j=3: 'c' == 'a', dp[3][3] = dp[2][2] + 1 = 2
return dp[3][3] == 2
```

Complexity Analysis

- **Time Complexity:** $O(M \cdot N)$ filling an $M \times N$ grid.
- **Space Complexity:** $O(M \cdot N)$ or $O(\min(M, N))$ space using rolling rows.

Follow-Up Interview Variants

How to reconstruct the actual LCS string? Backtrack through the DP table from (m, n) : if characters match, prepend to string and move diagonally; else move in the direction of the larger neighbor.

Problem 65: Edit Distance (Medium - Dynamic Programming)

Problem Statement

Given two strings `word1` and `word2`, return the minimum number of operations required to convert `word1` to `word2`. You have three operations permitted on a word: insert a character, delete a character, or replace a character.

Brute Force Approach & Complexity

Try all 3 operations at every character recursively: $O(3^{\max(M,N)})$.

Key Insight ("Aha!" Moment)

Classic Levenshtein 2D DP: if characters match (`word1[i-1] == word2[j-1]`), cost is 0: `dp[i][j] = dp[i-1][j-1]`. If they differ, take $1 + \min(\text{insert} : dp[i][j-1], \text{delete} : dp[i-1][j], \text{replace} : dp[i-1][j-1])$.

Optimal Python Solution

```
def editDistance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1)] * (m + 1)
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j - 1])
    return dp[m][n]
```

Edge-Case Asserts

```
assert editDistance("", "") == 0
assert editDistance("a", "a") == 0
assert editDistance("a", "b") == 1
assert editDistance("abc", "abc") == 0
assert editDistance("abc", "ab") == 1
assert editDistance("abc", "ac") == 1
assert editDistance("abc", "abc") == 0
```

Dry-Run Execution Trace

```
word1 = "abc", word2 = "abc"
m, n = 3, 3
dp = [[0] * 4] * 4
# i=1, j=1: 'a' == 'a' -> dp[1][1] = dp[0][0] = 0
# i=1, j=2: 'a' != 'b' -> dp[1][2] = 1 + min(dp[1][1], dp[0][2], dp[0][1]) = 1
# i=1, j=3: 'a' != 'c' -> dp[1][3] = 1 + min(dp[1][2], dp[0][3], dp[0][2]) = 2
# i=2, j=1: 'b' != 'a' -> dp[2][1] = 1 + min(dp[2][0], dp[1][1], dp[1][0]) = 1
# i=2, j=2: 'b' == 'b' -> dp[2][2] = dp[1][1] = 0
# i=2, j=3: 'b' != 'c' -> dp[2][3] = 1 + min(dp[2][2], dp[1][3], dp[1][2]) = 1
# i=3, j=1: 'c' != 'a' -> dp[3][1] = 1 + min(dp[3][0], dp[2][1], dp[2][0]) = 1
# i=3, j=2: 'c' != 'b' -> dp[3][2] = 1 + min(dp[3][1], dp[2][2], dp[2][1]) = 1
# i=3, j=3: 'c' == 'c' -> dp[3][3] = dp[2][2] = 0
return dp[3][3] = 0
```

Complexity Analysis

- **Time Complexity:** $O(M \cdot N)$ time filling the matrix.
- **Space Complexity:** $O(M \cdot N)$ or $O(\min(M, N))$ space with rolling row optimization.

Follow-Up Interview Variants

What if character weights differ (e.g. deletion costs 2, replacement costs 3)? Modify the constants inside the `min( ... )` branch accordingly.

Problem 66: Partition Equal Subset Sum (Medium - Dynamic Programming)

Problem Statement

Given an integer array `nums`, return `True` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `False` otherwise.

Brute Force Approach & Complexity

Generate all subsets and compare subset sums: $O(2^N)$.

Key Insight ("Aha!" Moment)

Total sum must be even; target sum for each subset is $\text{target} = \text{total} // 2$. This is classic **0/1 Knapsack**: can we pick a subset of elements that sums exactly to `target`? Maintain a hash set of reachable sums.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \cdot \text{target})$ pseudo-polynomial time.
- **Space Complexity:** $O(\text{target})$ space to store reachable sums up to target.

Follow-Up Interview Variants

How to optimize space to a bitset? Represent the DP state with a single integer: `dp |= (dp << num)`; check if bit at `target` is set in $O(N \cdot \text{target} / 64)$.

Problem 67: Jump Game (Medium - Dynamic Programming)

Problem Statement

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position. Return `True` if you can reach the last index, or `False` otherwise.

Brute Force Approach & Complexity

Recursively explore every jump choice from current position: $O(2^N)$.

Key Insight ("Aha!" Moment)

Maintain the maximum index reachable so far `max_reach`. As you scan $i = 0 \dots N - 1$, if $i > \text{max_reach}$, you are stranded and cannot proceed. Otherwise, update $\text{max_reach} = \max(\text{max_reach}, i + \text{nums}[i])$.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass over array.
- **Space Complexity:** $O(1)$ constant auxiliary memory.

Follow-Up Interview Variants

How to compute the MINIMUM number of jumps needed to reach the end (Jump Game II)? Use BFS window intervals `[curr_end, farthest]` in $O(N)$ time and $O(1)$ space.

Problem 68: Insert Interval (Medium - Intervals)

Problem Statement

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [start_i, end_i]` sorted in ascending order by `start_i`. You are also given an interval `newInterval`. Insert `newInterval` into `intervals` such that `intervals` is still sorted and non-overlapping (merge overlapping intervals if necessary).

Brute Force Approach & Complexity

Append `newInterval` to `intervals`, sort in $O(N \log N)$, and run merge intervals.

Key Insight ("Aha!" Moment)

Since the input is already sorted, execute in three linear phases: 1) Add all intervals ending before `newInterval.start`, 2) Merge all intervals overlapping with `newInterval`, 3) Add all intervals starting after `newInterval.end`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single linear pass.
- **Space Complexity:** $O(N)$ output array memory.

Follow-Up Interview Variants

Could binary search be used to find insertion positions? Yes, binary search finds start and end indices in $O(\log N)$, but splicing the array still requires $O(N)$ time.

Problem 69: Merge Intervals (Medium - Intervals)

Problem Statement

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Brute Force Approach & Complexity

Graph connected components: represent intervals as nodes, overlap as edges, and find connected components. Time: $O(N^2)$.

Key Insight ("Aha!" Moment)

Sort intervals by start time. Iterate through sorted intervals: if the current interval starts before or at the end of the previous interval (`curr[0] ≤ prev[1]`), merge them by extending `prev[1] = max(prev[1], curr[1])`. Otherwise, append `curr` as a new interval.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log N)$ dominated by sorting.
- **Space Complexity:** $O(N)$ for output list and sorting memory.

Follow-Up Interview Variants

What if input intervals are an unbounded data stream? Maintain an Interval Tree or Segment Tree to support dynamic insertion and merging.

Problem 70: Non-overlapping Intervals (Medium - Intervals)

Problem Statement

Given an array of intervals `intervals` where `intervals[i] = [start_i, end_i]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Brute Force Approach & Complexity

Generate all subsets of intervals and check if they are pairwise non-overlapping. Time: $O(2^N)$.

Key Insight ("Aha!" Moment)

This is Interval Scheduling Maximization: to maximize the count of mutually compatible intervals, greedily pick intervals that **end earliest**. Sort by end time, and greedily remove any interval that starts before the previously kept interval ends.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log N)$ sorting time.
- **Space Complexity:** $O(1)$ auxiliary memory.

Follow-Up Interview Variants

Why does sorting by END time work greedily while sorting by START time does not? An interval ending early leaves maximum room for future intervals.

Problem 71: Meeting Rooms (Easy - Intervals)

Problem Statement

Given an array of meeting time intervals where `intervals[i] = [start_i, end_i]`, determine if a person could attend all meetings without overlap.

Brute Force Approach & Complexity

Compare all pairs (i, j) and check if they overlap. Time: $O(N^2)$, Space: $O(1)$.

Key Insight ("Aha!" Moment)

Sort meetings by start time. A person can attend all meetings if and only if every meeting starts after or at the exact time the previous meeting concludes (`intervals[i][0] ≥ intervals[i-1][1]`).

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log N)$ sorting time.
- **Space Complexity:** $O(1)$ auxiliary space.

Follow-Up Interview Variants

What if meeting times are given as timestamps with timezones? Convert all timestamps into UTC Unix epoch milliseconds before sorting.

Problem 72: Meeting Rooms II (Medium - Intervals)

Problem Statement

Given an array of meeting time intervals `intervals` where `intervals[i] = [start_i, end_i]`, return the minimum number of conference rooms required.

Brute Force Approach & Complexity

For every minute in time, count how many meetings are active and find the peak. Time: $O(T \cdot N)$ where T is time range.

Key Insight ("Aha!" Moment)

Sort meetings by start time and maintain a **Min-Heap** of meeting end times. When a new meeting starts, check if the room with the earliest end time has freed up (`heap[0] ≤ meeting[0]`). If so, reuse that room; otherwise, allocate a new room.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N \log N)$ time for sorting and heap operations.
- **Space Complexity:** $O(N)$ heap memory.

Follow-Up Interview Variants

Could this be solved with the Line Sweep Algorithm? Yes, split intervals into start (+1) and end (-1) timestamp events, sort events, and track the maximum running prefix sum in $O(N \log N)$.

Problem 73: Number of 1 Bits (Easy - Bit Manipulation)

Problem Statement

Given a positive integer `n`, write a function that returns the number of set bits (1s) it has (also known as the Hamming weight).

Brute Force Approach & Complexity

Loop through all 32 bits and count how many times `n & 1` is 1 by shifting `n >>= 1`. Takes 32 iterations.

Key Insight ("Aha!" Moment)

Brian Kernighan's Algorithm: the operation `n & (n - 1)` clears the lowest set bit of `n`. Looping `while n: n &= (n - 1)` executes in exactly $O(K)$ iterations where K is the number of 1-bits.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(K)$ where K is number of set bits (at most 32 or 64 operations).
- **Space Complexity:** $O(1)$ constant auxiliary memory.

Follow-Up Interview Variants

How does CPython / hardware implement this? Modern x86 processors have a dedicated `POPCNT` instruction that executes in 1 clock cycle.

Problem 74: Counting Bits (Easy - Bit Manipulation)

Problem Statement

Given an integer `n`, return an array `ans` of length `n + 1` such that for each `i` ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of `i`. Must run in $O(N)$ linear time.

Brute Force Approach & Complexity

Compute `hamming_weight(i)` independently for each number from 0 to n : $O(N \log N)$.

Key Insight ("Aha!" Moment)

Dynamic Programming with bit shifting: right shifting i by 1 (`i >> 1`) removes its least significant bit. The number of 1-bits in i is simply `dp[i >> 1] + (i & 1)`.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ linear time single pass.
- **Space Complexity:** $O(1)$ auxiliary memory (output array excluded).

Follow-Up Interview Variants

Could you also compute this using Most Significant Bit (MSB)? Yes, `dp[i] = 1 + dp[i - offset]` where `offset` doubles whenever i reaches a power of 2.

Problem 75: Missing Number (Easy - Bit Manipulation)

Problem Statement

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array.

Brute Force Approach & Complexity

Sort array in $O(N \log N)$ or use a hash set in $O(N)$ space.

Key Insight ("Aha!" Moment)

XOR self-cancellation: $a \oplus a = 0$ and $a \oplus 0 = a$. XORing all expected numbers $0 \dots n$ with all actual numbers in `nums` causes every present number to cancel out, leaving strictly the missing number in $O(1)$ space.

Optimal Python Solution

Edge-Case Asserts

Dry-Run Execution Trace

Complexity Analysis

- **Time Complexity:** $O(N)$ single pass over array.
- **Space Complexity:** $O(1)$ constant space without risk of integer overflow.

Follow-Up Interview Variants

Could Gauss's summation formula $\frac{n(n+1)}{2} - \sum nums$ be used instead? Yes, though in fixed-width integers (C++/Java) XOR is safer as it eliminates integer overflow risks.

Further Reading

- [CP-Algorithms: graphs](#)
- [Python heapq](#)
- [Python bisect](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Run `python practice/run_tests.py 12 --stub` on an unimplemented stub: what do you see?
- ▶ 2. What does `--fuzz` do?
- ▶ 3. Why trust a fuzz check more than three hand-written asserts?
- ▶ 4. Pick one problem: state the brute force, the insight and the complexity without looking.